impl PrivacyProfile::from_pairs

Michael Shoemate

May 1, 2026

This proof resides in **"contrib"** because it has not completed the vetting process.

1 Hoare Triple

Precondition

Compiler-verified

- Argument `pairs` of type `Vec<(f64, f64)>`

Caller-verified

None

Pseudocode

```python
import bisect

def privacyprofile_from_pairs(I: Intent, pairs: list[tuple[float, float]]) -> PrivacyProfile:
    if not pairs:
        raise ValueError("privacy profile must be defined by at least one pair")

    # Validate pairs
    for eps, delta in pairs:
        if not (isinstance(eps, float) and eps >= 0 and eps != float('inf')):
            raise ValueError("epsilon must be finite and non-negative")
        if not (isinstance(delta, float) and 0 <= delta <= 1):
            raise ValueError("delta must be finite and in [0, 1]")

    # Sort increasing on epsilon, then use the intent-specific conservative tie-break.  #
    if I.CALIBRATION:
        pairs.sort(key=lambda x: (x[0], x[1]))
    else:
        pairs.sort(key=lambda x: (x[0], -x[1]))

    # For duplicate epsilons, keep the first value after the intent-specific sort.
    deduplicated = []  #
    for eps, delta in pairs:
        if not deduplicated or deduplicated[-1][0] != eps:
            deduplicated.append((eps, delta))

    pairs = deduplicated  #
    epsilon_grid, delta_grid = zip(*pairs)
    epsilon_grid = list(epsilon_grid)
```

```
29      delta_grid = list(delta_grid)
30
31      # Check that delta is monotonically nonincreasing
32      for window in zip(delta_grid, delta_grid[1:]):  #
33          if window[0] < window[1]:
34              raise ValueError(
35                  "delta values must be monotonically nonincreasing as epsilon increases"
36              )
37
38      def profile(epsilon: float) -> float:
39          if not (isinstance(epsilon, float) and epsilon >= 0 and epsilon == epsilon):
40              raise ValueError("epsilon must be a finite non-negative number")
41
42          # Use the intent-specific side of each staircase jump.  #
43          # Return the intent-specific endpoint value.  #
44          if I.CALIBRATION:
45              # First sampled epsilon strictly greater than or equal to epsilon.
46              idx = bisect.bisect_left(epsilon_grid, epsilon)
47              if idx == len(delta_grid):
48                  return 0.0
49              return delta_grid[idx]
50          else:
51              # Last sampled epsilon less than or equal to epsilon.
52              idx = bisect.bisect_right(epsilon_grid, epsilon)
53              if idx == 0:
54                  return 1.0
55              return delta_grid[idx - 1]
56
57      return PrivacyProfile(profile, epsilon_grid=epsilon_grid)
```

Postcondition

Theorem 1.1. The pseudocode returns a privacy profile $p : \mathbb{R}_{\geq 0} \to [0, 1]$ such that:

1. For `Reporting`, p is the right-continuous staircase that returns the δ_i associated with the largest sampled $\epsilon_i \leq \epsilon$, or 1 before the first sample.

2. For `Calibration`, p is the left-continuous staircase that returns the δ_i associated with the smallest sampled $\epsilon_i \geq \epsilon$, or 0 after the last sample.

3. Duplicate epsilons are resolved conservatively with respect to the intent: maximum δ for `Reporting`, minimum δ for `Calibration`.

Proof. Let $\sigma = \text{sort}(\text{pairs})$ be the list after sorting by ϵ ascending and by the intent-specific delta order (line 14). Let $\sigma' = \text{dedup}(\sigma)$ be the result after removing duplicates on ϵ (lines 21–26), retaining the first delta for each unique ϵ. Let $\epsilon_1, \ldots, \epsilon_n$ and $\delta_1, \ldots, \delta_n$ be the epsilon and delta values in σ'.

Sorting and Deduplication For `Reporting`, sorting by $(\epsilon, -\delta)$ ensures that among all pairs with the same ϵ, the largest δ appears first. Since duplicates are removed greedily, only the maximum δ per unique ϵ survives. This is conservative for reporting because larger δ is weaker and therefore does not overstate privacy. For `Calibration`, sorting by (ϵ, δ) ensures that the smallest δ appears first and survives. This is conservative for calibration because smaller δ is stronger and therefore mechanisms calibrated against the profile satisfy at least the requested guarantee.

Monotonicity Invariant Let I be the property: $\delta_i \geq \delta_{i+1}$ for all $1 \leq i < n$. The monotonicity check (line 32) ensures I holds. If I fails, an error is returned. Property I is necessary for the result to be a valid privacy profile: as privacy budget (epsilon) increases, privacy guarantees (delta) cannot improve.

Correctness of Query Evaluation For `Reporting`, let $i^* = \max\{i : \epsilon_i \leq \epsilon\}$, or $i^* = 0$ if no such index exists. The binary search (line 42) computes this position, and the profile returns (line 43):

$$p(\epsilon) = \begin{cases} 1 & \text{if } i^* = 0 \\ \delta_{i^*} & \text{otherwise} \end{cases}$$

For `Calibration`, let $j^* = \min\{j : \epsilon_j \geq \epsilon\}$, or no index if ϵ is larger than all samples. The binary search (line 42) computes this position, and the profile returns:

$$p(\epsilon) = \begin{cases} \delta_{j^*} & \text{if } j^* \text{ exists} \\ 0 & \text{otherwise.} \end{cases}$$

Conservatism The reporting staircase uses the last known guarantee that applies at or below the query epsilon, returning 1 before any sampled guarantee applies. This rounds privacy loss upward because larger δ is weaker. The calibration staircase uses the next known guarantee at or above the query epsilon, returning 0 beyond the last sample. This rounds privacy loss downward because smaller δ is stronger. Thus the constructed profile follows the intent semantics. □